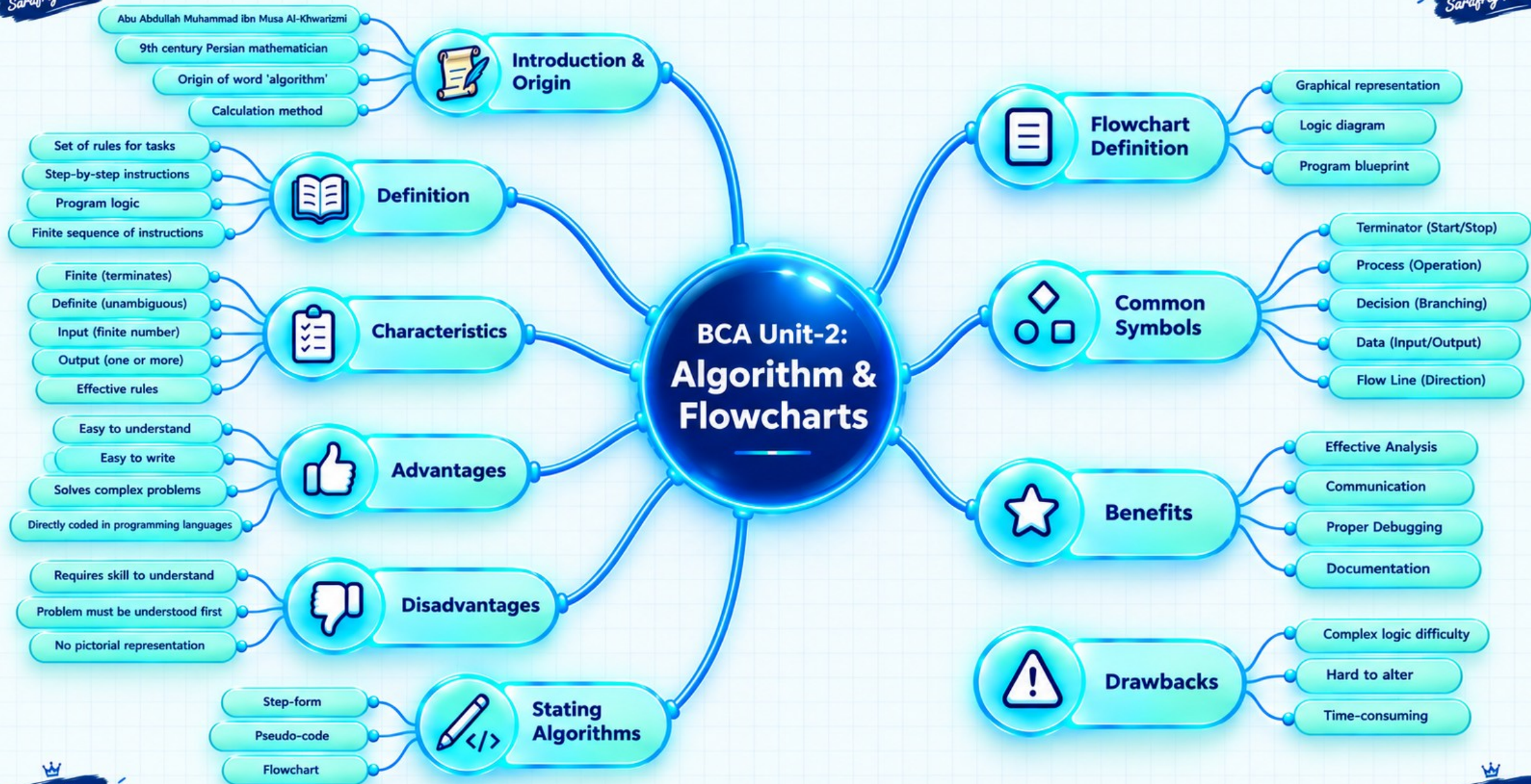


UNIT - 2

Algorithm & Flowcharts

Concepts • Characteristics • Flowchart Symbols • C Program Examples



2.1 ALGORITHM



Introduction & Origin

Introduction

The word “algorithm” is the name of Abu Abdullah Muhammad ibn Musa Al-Khwarizmi who was a Persian mathematician in 9th century. An algorithm is a set of rules which can be used to do some task either manually or electronically (computer system) is known as an algorithm. Most commonly, this term is used with reference to computers. Generally algorithms are used to provide solutions of different problems. Algorithms provide effective methods for problem solving. It solves a problem with a finite sequence of instructions.

Origin

The term “algorithm” is derived from the name of the great Persian mathematician Abu Abdullah Muhammad ibn Musa Al-Khwarizmi, who wrote a book on algebra in the 9th century. His name is often considered the origin of the word “algorithm”.

★ Key Points

- Algorithm is the name of Abu Abdullah Muhammad ibn Musa Al-Khwarizmi.
- It is a set of rules to solve a task.
- Commonly used in computers for problem solving.
- Provides effective methods and finite sequence of instructions.



Al-Khwarizmi



2.1.2 Meaning and Definition



Definition

The term “Algorithm” is taken from a Latin word “algorismus”. The meaning of this word is “calculation method”. To solve a problem specific set of instructions are used, this set of instruction is known as algorithm. For a given problem it is a step by step instruction for how to reach at the solution of that problem. Basically algorithm shows the logic of the program. An algorithm is defined as a finite set of well defined instructions for problem solving. It takes a valid input and produces desired output in a finite amount of time. It is illustrated in figure 2.1.

★ Key Points

- An algorithm is generally used for calculation and processing data using a sequence of finite instructions.
- An algorithm provides step-by-step procedure for problem solving.
- According to Stone and Knuth “an algorithm is a set of rules that precisely defines a sequence of operations in such a way that each rule is effective and definite and that the sequence terminates in a finite time.” An algorithm has a specific format which is shown in figure 2.2.



Figure 2.1: Algorithm

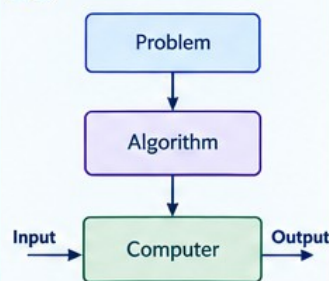


Figure 2.1: Algorithm



An algorithm is generally used for calculation and processing data using a sequence of finite instruction set.



Disadvantages of Algorithms

- Occupy more skill to understand it.
- Problem should be understand before proceed for algorithm.
- Cannot be represented in pictorial form.



Examples

Example 1: Find the area and circumference of a circle of radius r.

Algorithm

- Step 1: Read Radius of Circle: as R
- Step 2: Assign Value of PI = 3.14
- Step 3: Calculate Area = $PI * R * R$
- Step 4: Calculate Circumference Circum = $2 * PI * R$
- Step 5: Print Value of Area and Circum
- Step 6: END / STOP



Example 2

Find the area and circumference of a circle of radius r.

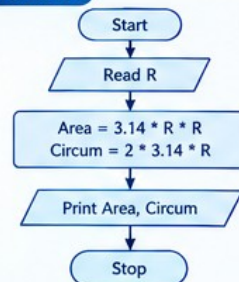


Algorithm

- Step 1: Read Radius of Circle as R
- Step 2: Assign Value of PI = 3.14
- Step 3: Calculate Area = $PI * R * R$
- Step 4: Calculate Circumference Circum = $2 * PI * R$
- Step 5: Print Value of Area and Circum
- Step 6: END / STOP



Flowchart



C Program

```
#include <stdio.h>
#include <math.h>
int main() {
    float R, PI = 3.14, Area, Circum;
    printf("Enter radius of circle: ");
    scanf("%f", &R);
    Area = PI * R * R;
    Circum = 2 * PI * R;
    printf("Area = %.2f\n", Area);
    printf("Circumference = %.2f\n", Circum);
    return 0;
}
```



Output

Enter radius of circle: 5
Area = 78.50
Circumference = 31.40



Explanation

The program takes the radius as input, calculates the area and circumference using the given formulas and displays the result.



C Program

```
#include <stdio.h>
#include <math.h>
int main() {
    float R, PI = 3.14, Area, Circum;
    printf("Enter radius of circle: ");
    scanf("%f", &R);
    Area = PI * R * R;
    Circum = 2 * PI * R;
    printf("Area = %.2f\n", Area);
    printf("Circumference = %.2f\n", Circum);
    return 0;
}
```



Output

Enter radius of circle: 5
Area = 78.50
Circumference = 31.40



Explanation

The program takes the radius as input, calculates the area and circumference using the given formulas and displays the result.

2.1 Characteristics of Algorithm

1



Finite

An algorithm must have finite number of steps after which it would be terminated.

2



Definite

Algorithms must not have any ambiguity. Each step of the algorithm must be well-defined and unambiguous.

3



Input

An algorithm can have finite number of inputs.

4



Output

An algorithm can have one or more outputs.

5



Effective

An algorithm must be effective and can be implemented with a finite amount of time.

1



Advantages of Algorithms

- 1 Provides step-by-step procedure to understand easily.
- 2 Algorithm is easy to write.
- 3 Solves those problems which are complex and not solve by simple methods.
- 4 Algorithm in general, can be directly coded in a programming language.

2



Disadvantages of Algorithms

- 1 Occupy more skill to understand it.
- 2 Problem should be understand before proceed for algorithm.
- 3 Cannot be represented in pictorial form.



Example 8: Algorithm of making tea.

Algorithm

- START**
- Step 1:** Assemble tools – cups, saucers, spoons, milk, kettle and teapot.
- Step 2:** Count the number of persons (one or two or more).
- Step 3:** Add 1 spoon of tea leaves to teapot (and 1/2 spoon for every extra person).
- Step 4:** Add water to kettle.
- Step 5:** Turn on heat source.
- Step 6:** Heat kettle water to boiling.
- Step 7:** Add water to teapot.
- Step 8:** Leave tea to brew.
- Step 9:** Pour tea into teacups.
- Step 10:** If milk wanted, add milk.
- Step 11:** If sugar wanted, add sugar.
- Step 12:** Drink and enjoy!
- Step 13:** STOP

C Program

```
void main() {
    int n;
    printf("Enter number of persons: ");
    scanf("%d", &n);

    printf("Enter 1/2 spoon of tea leaves: ");
    scanf("%d", &n); // (simplified input)

    printf("Add water to kettle.\n");
    printf("Turn on heat source.\n");
    printf("Heat kettle water to boiling.\n");
    printf("Add water to teapot.\n");
    printf("Leave tea to brew.\n");
    printf("Pour tea into teacups.\n");
    printf("If milk wanted, add milk.\n");
    printf("If sugar wanted, add sugar.\n");
    printf("Drink and enjoy!\n");
}
```

Explanation

This program reads the number of persons and follows the same steps given in the algorithm to make tea. It only displays the steps on the screen.

Example 9: Algorithm to find the largest of three numbers.

Algorithm

- START**
- Step 1:** Read first number.
- Step 2:** Read second number.
- Step 3:** Read third number.
- Step 4:** Compare and find the largest.
- Step 5:** Display the largest number.
- Step 6:** STOP

C Program

```
void main() {
    int a, b, c, largest;
    printf("Enter first number: ");
    scanf("%d", &a);
    printf("Enter second number: ");
    scanf("%d", &b);
    printf("Enter third number: ");
    scanf("%d", &c);

    largest = a;
    if (b > largest) largest = b;
    if (c > largest) largest = c;
    printf("Largest number is: %d\n",
    return 0;
}
```

Explanation

The program reads three numbers and compares them using conditional statements to find the largest number. The result is then displayed on the screen.

Example 10: Write the steps for calculating the average of two numbers.

Algorithm

- START**
- Step 1:** Read principal, rate and time.
- Step 2:** Calculate $P = P \times R \times T$.
- Step 3:** Calculate $S = Q / 100$.
- Step 4:** Display S.
- Step 5:** STOP

Explanation

The program takes principal, rate and time as input, calculates the simple interest using the given formula and displays the result.

C Program

```
#include <stdio.h>
void main() {
    float prin, num1, num2, avg;
    printf("Enter principal: ");
    scanf("%f", &num1);
    printf("Enter rate: ");
    scanf("%f", &num2);
    avg = (num1 + num2) / 2.0;
    printf("Average = %.2f\n", avg);
    return 0;
}
```

Example 11: Write an algorithm to lock an open room.

Algorithm

- Step 1:** Take the key of the lock.
- Step 2:** Closed the door.
- Step 3:** Put the key in the lock.
- Step 4:** Turn the key clockwise direction.
- Step 5:** Remove the key from the lock.
- Step 6:** STOP

C Program

```
void main() {
    printf("This is a physical action.");
    printf("Locking an open room cannot
    be implemented in C.");
    return 0;
}
```

Explanation

This is a real-world procedure to lock a room. It involves physical actions and cannot be implemented as a C program.



Example 12: Finding the leap Year.

Question Write a program to check whether a given year is a leap year or not.

Algorithm

START

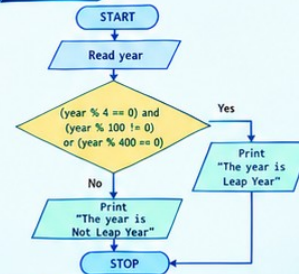
Step 1: Read year.

Step 2: If (year % 4 == 0) and (year % 100 != 0) or (year % 400 == 0) then print "The year is Leap Year";

Step 3: Else print "The year is Not Leap Year".

STOP

Flowchart



C Program

```
#include <stdio.h>
void main() {
    int year;
    printf("Enter year: ");
    scanf("%d", &year);
    if ((year%4==0 && year%100!=0) || (year%400==0))
        printf("The year is Leap Year");
    else
        printf("The year is Not Leap Year");
}
```

Output

Enter year: 2024
The year is Leap Year

Explanation

The program checks the given year for the leap year condition using the given formula and displays the result.

Example 13: Addition of Two Integers.

Question Write a program to add two integers and display the sum.

Algorithm

START

Step 1: Initialize a, b, and z.

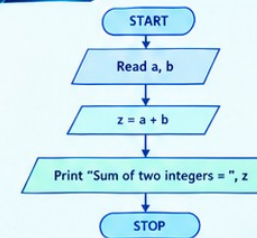
Step 2: Read a, b.

Step 3: z = a + b.

Step 4: Print "Sum of two integers".

Step 5: **STOP**

Flowchart



C Program

```
#include <stdio.h>
void main() {
    int a, b, z;
    printf("Enter first number: ");
    scanf("%d", &a);
    printf("Enter second number: ");
    scanf("%d", &b);
    z = a + b;
    printf("Sum of two integers = %d", z);
}
```

Output

Enter first number: 52
Enter second number: 30
Sum of two integers = 82

Explanation

The program reads two integers, adds them and displays the sum using the given formula.

Example 14: To choose the largest of two distinct numbers.

Algorithm Write the program to find the largest of two distinct numbers.

Algorithm

START

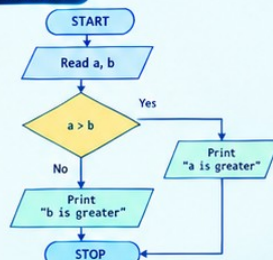
Step 1: Initialize a, b.

Step 2: Read a and b.

Step 3: If (a > b) print "a is greater"; else print "b is greater".

Step 4: **STOP**

Flowchart



C Program

```
#include <stdio.h>
void main() {
    int a, b;
    printf("Enter first number: ");
    scanf("%d", &a);
    printf("Enter second number: ");
    scanf("%d", &b);
    if (a > b)
        printf("a is greater");
    else
        printf("b is greater");
}
```

Output

Enter first number: 45
Enter second number: 28
a is greater

Explanation

The program compares the two numbers using a conditional statement and displays the larger number.

Example 15: Find the sum of first n natural numbers.

Algorithm Write a program to find the sum of first n natural numbers.

Algorithm

START

Step 1: Initialize SUM = 0 and i = 0.

Step 2: Read N.

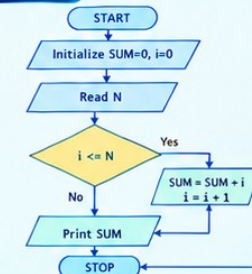
Step 3: Check i <= N. If Yes Then SUM = SUM + i.

Step 4: Increment i = i + 1.

Step 5: Else Print SUM.

Step 6: **STOP**

Flowchart



C Program

```
#include <stdio.h>
void main() {
    int n, i, sum=0;
    printf("Enter n: ");
    scanf("%d", &n);
    for(i=1; i<=n; i++)
        sum = sum + i;
    printf("Sum = %d", sum);
}
```

Output

Enter n: 50
Sum = 1275

Explanation

The program adds numbers from 1 to n using a loop and displays the sum.

2.2 Different Ways of Stating Algorithms

There are three ways of stating algorithms. These are as follows:

-  **1. Step-form**
-  **2. Pseudo-code**
-  **3. Flowchart**

2.2.1 Step-form

It is the simplest form of algorithm and it is also easy to learn. It comprises of numbered steps or points. It is written in a clear and simple manner so that the logic can be easily understood.

Conventions for Step-form

- START and STOP**
Two statements START and STOP are logically enclosed with every algorithm.
- INPUT or READ**
INPUT or READ statements are used to receive the information from the user.
- PRINT**
PRINT statement is used to show or display the variables value or the user's contents.
- Arithmetic Operators**
Following are the arithmetic operators used in the expression:
= (Assignment) + (Addition) * (Multiplication)
- (Subtraction) / (Division)
- Relational Operators**
Following are the relational operators used in the expression:
> < >= <= = !=
- Logical Operators**
Following are the logical operators used in the expression:
AND OR NOT

Operator Reference Table

Category	Operators	Category	Operators
Assignment	=	Greater than	>
Addition	+	Less than	<
Multiplication	*	Greater than or equal to	>=
Subtraction	-	Less than or equal to	<=
Division	/	Equality	=
		Not equal to	!=

Note

Equality vs Assignment
= is used for assignment (putting a value into a variable).
while == is used for equality (checking if two values are equal).

2.2.2 Pseudo code

The high-level description of the computer programming algorithm in a compact and informal form is known as pseudo-code. It is not proposed for the machine reading, also uses the programming language's structural conventions.

Pseudo Code Representation / Conventions for Pseudo Code

- It should be easy to understand and clear.
- Always assign a valid name to the pseudo code procedure.
- For each line of code always use the line number.
- In block structure use the suitable indentation for every statement.
- For the flow control, the following are used:
i) 'AND' Conjunction ii) 'OR' Disjunction iii) 'NOT' Negation.
- The 'a = b' statement checks the equality and 'a ← b' checks the assignment.
- The maximum number of lines should be as small as possible.
- Use meaningful variable names.
- The pseudo-code should be written in simple and clear language.

Example 19: Find the sum of two numbers.

Question

Find the sum of two numbers.

Algorithm

- Step 1: START
Step 2: Read A, B
Step 3: C = A + B
Step 4: Print C
Step 5: STOP

C Program

```
#include <stdio.h>
int main() {
    int a, b, c;
    printf("Enter A and B: ");
    scanf("%d %d", &a, &b);
    c = a + b;
    printf("Sum = %d\n", c);
    return 0;
}
```

Output

Enter A and B: 5 7
Sum = 12

Explanation

The program reads two numbers and adds them. The result is displayed as the sum.

Example 20: Find the largest among three numbers.

Question

Print the largest among three numbers.

Algorithm

- Step 1: START
Step 2: Read A, B, C
Step 3: MAX = A
Step 4: If B > MAX then MAX = B
Step 5: If C > MAX then MAX = C
Step 6: Print MAX
Step 7: STOP

C Program

```
#include <stdio.h>
int main() {
    int a, b, c, max;
    printf("Enter three numbers: ");
    scanf("%d %d %d", &a, &b, &c);
    max = a;
    if (b > max) max = b;
    if (c > max) max = c;
    printf("Largest number = %d\n", max);
    return 0;
}
```

Output

Enter three numbers:
10 25 7
Largest number = 25

Explanation

The program compares the three numbers and updates MAX accordingly to get the largest number.

Example 21: arrayMax(a[1..n], n), find the maximum element in an array.

Question

Find the maximum element in an array of size n.

Algorithm

- Step 1: START
Step 2: Read n and array elements
Step 3: max = a[0]
Step 4: For i = 1 to n-1
 If a[i] > max then max = a[i]
Step 5: Print max
Step 6: STOP

C Program

```
#include <stdio.h>
int main() {
    int a[100], n, max, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
    max = a[0];
    for (i = 1; i < n; i++) {
        if (a[i] > max) max = a[i];
    }
    printf("Maximum element = %d\n", max);
    return 0;
}
```

Output

Enter number of elements: 5
Enter elements: 12 45 7 23 6
Maximum element = 45

Explanation

The program reads n numbers from the user and compares each element with the current maximum. The largest element is displayed as the result.



Flowcharts



Definition of Flowchart

A flowchart is a graphical representation of an algorithm or program flow. It uses standard symbols to represent different steps of a problem. It helps in understanding, testing and debugging a program.



Short Definition

"A flowchart is a graphical representation of an algorithm or program flow. It uses standard symbols to represent different steps of a problem. It helps in understanding, testing and debugging a program."



Guidelines for Drawing a Flowchart

- 1 Arrange all requirements in logical order.
- 2 Avoid ambiguity and keep the flowchart clear and neat.
- 3 Flow should normally be left-to-right or top-to-bottom.
- 4 From a process symbol, only one flow line should come out.
- 5 A decision symbol can have two or more output flow lines.
- 6 A terminal symbol generally has only one flow line.
- 7 Use standard symbols.
- 8 Use connectors to reduce complexity.
- 9 A flowchart should always have a logical start and finish.
- 10 Test the flowchart using simple test data.



Common Flowchart Symbols

No.	Symbol	Name	Use
1		Process	Shows an operation or action.
2		Terminator	Shows Start or Stop.
3		Decision	Shows a condition or question.
4		Delay	Shows waiting period.
5		Predefined Process	Shows a sub-process.
6		Data (I/O)	Shows input and output.
7		Document	Shows a document or report.
8		Display	Shows machine display.
9		Manual Input	Shows manual input.
10		Connector	Connects different points.
11		Off-Page Connector	Connects to another page.
12		Flow Line	Shows direction of flow.



Advantages of Flowcharts

- 1 **Communication** – Helps to communicate program logic.
- 2 **Effective Analysis** – Makes program analysis easier.
- 3 **Documentation** – Provides good program documentation.
- 4 **Debugging** – Helps in finding and correcting errors.
- 5 **Program Maintenance** – Makes program maintenance easier.



Short Trick

C-A-D-D-M

Communication → Analysis → Documentation
→ Debugging → Maintenance



Reasons for Drawing a Flowchart

- 1 It gives a visual representation of a program.
- 2 It is language independent.
- 3 It makes the logic of an algorithm easy to understand.
- 4 It helps to find logical errors before coding.
- 5 It helps in solving complex problems easily.
- 6 It is useful for program documentation.



Short Trick

C-A-D-D-M

Communication → Analysis → Documentation
→ Debugging → Maintenance



Disadvantages of Flowcharts

- 1 **Complex Logic** – Complex problems can make flowcharts difficult.
- 2 **Alterations and Modifications** – Changes may require redrawing the flowchart.
- 3 **Reproduction** – Reproducing a flowchart can be difficult.
- 4 **Time-Consuming** – Drawing symbols takes more time.
- 5 **Subjective** – Different programmers may draw different flowcharts for the same problem.



Short Trick

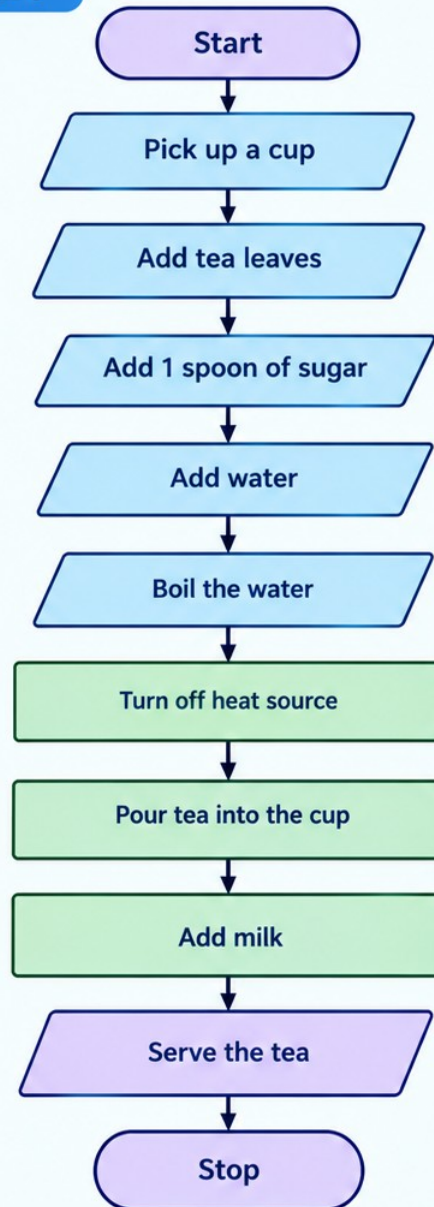
C-A-R-T-S

Complex Logic → Alterations → Reproduction
→ Time → Subjective

Example 22 : Making a Cup of Tea



Flowchart



Flowchart Symbols Used

Symbol	Name	Use
	Terminator	Start or Stop.
	Input / Output	Read input or show output.
	Process	Shows an operation or action.
	Flow Line	Shows direction of flow.



Explanation (Step by Step)

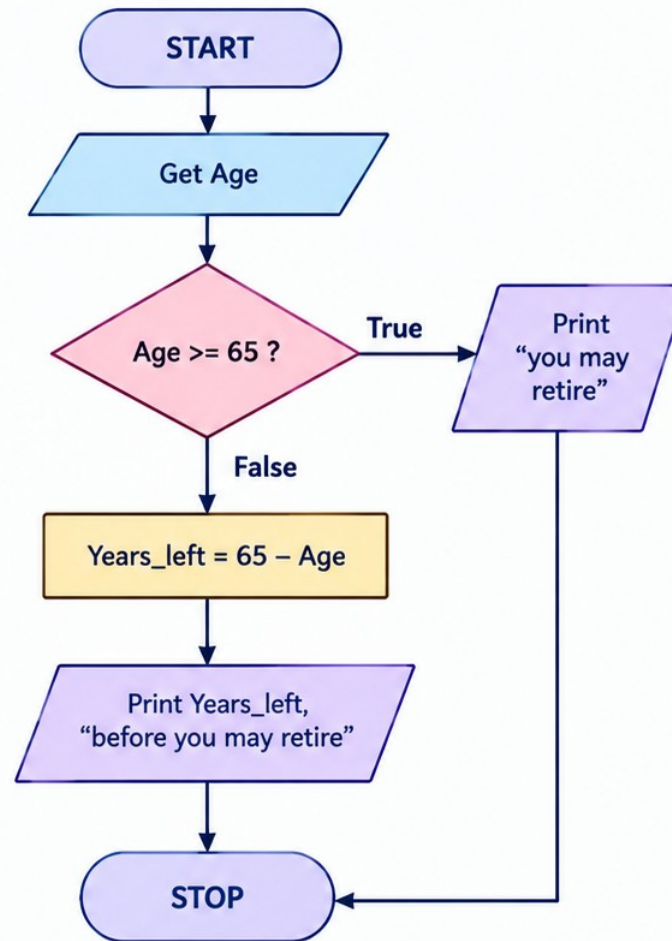
- 1 Start the process.
- 2 Pick up a cup.
- 3 Add tea leaves to the cup.
- 4 Add 1 spoon of sugar.
- 5 Add water.
- 6 Boil the water.
- 7 Turn off the heat source.
- 8 Pour the tea into the cup.
- 9 Add milk.
- 10 Serve the tea.
- 11 Stop.



Note:

This flowchart shows the step-by-step process of making a cup of tea using standard flowchart symbols.

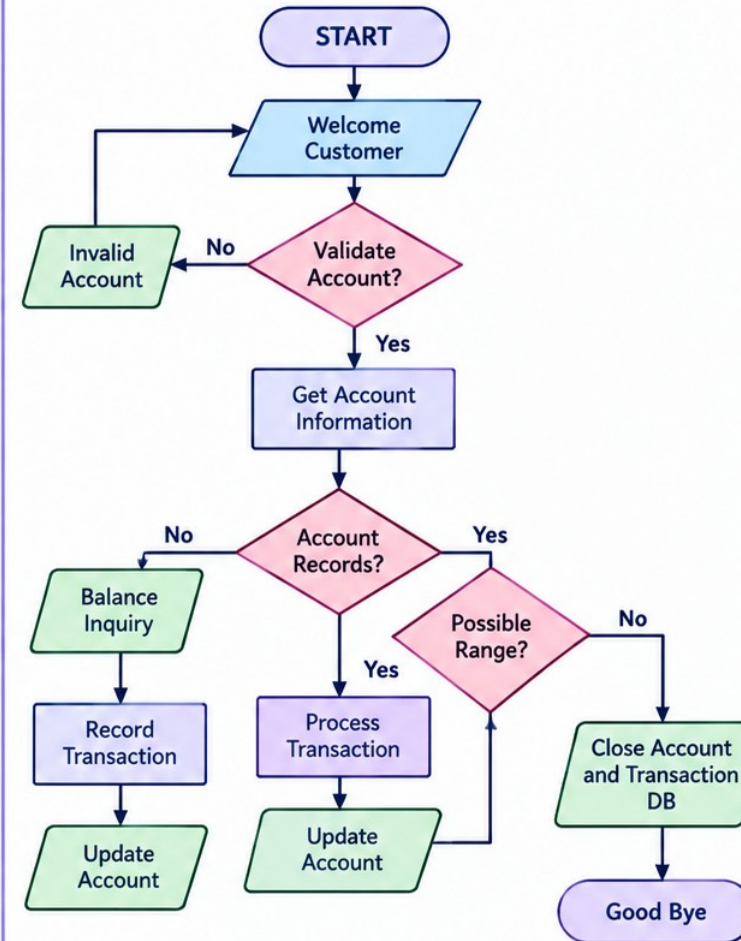
Example 23 : Checking age for Retirement



Explanation

1. Start and get the age.
2. Check if age is greater than or equal to 65.
3. If yes, print "you may retire" and stop.
4. If no, calculate remaining years (65 - age).
5. Print the remaining years and stop.

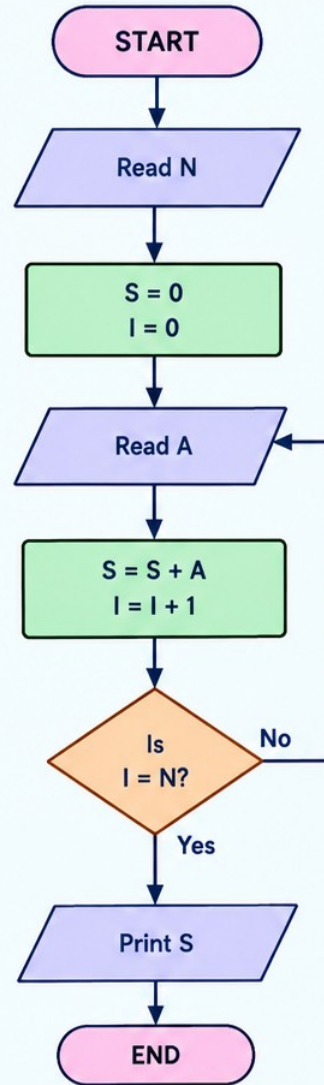
Example 24 : Process of ATM machine



Explanation

1. Start and welcome the customer.
2. Validate the account. If invalid, show "Invalid Account" and return.
3. If valid, get account information.
4. Check if account records exist.
5. If no, do balance inquiry → record transaction → update account.
6. If yes, check possible range → process transaction → update account.
7. Finally, close account and transaction DB and say Good Bye.

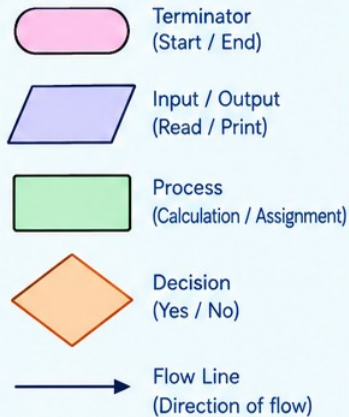
Example 25: Flowchart to Find the Sum of N Given Numbers



Steps:

1. Start the program.
2. Read the value of N (number of terms).
3. Set $S = 0$ and $I = 0$.
4. Read a number A.
5. Add A to S and increment I by 1.
6. Check if $I = N$.
 - If No, repeat from step 4.
 - If Yes, go to step 7.
7. Print the sum S.
8. End the program.

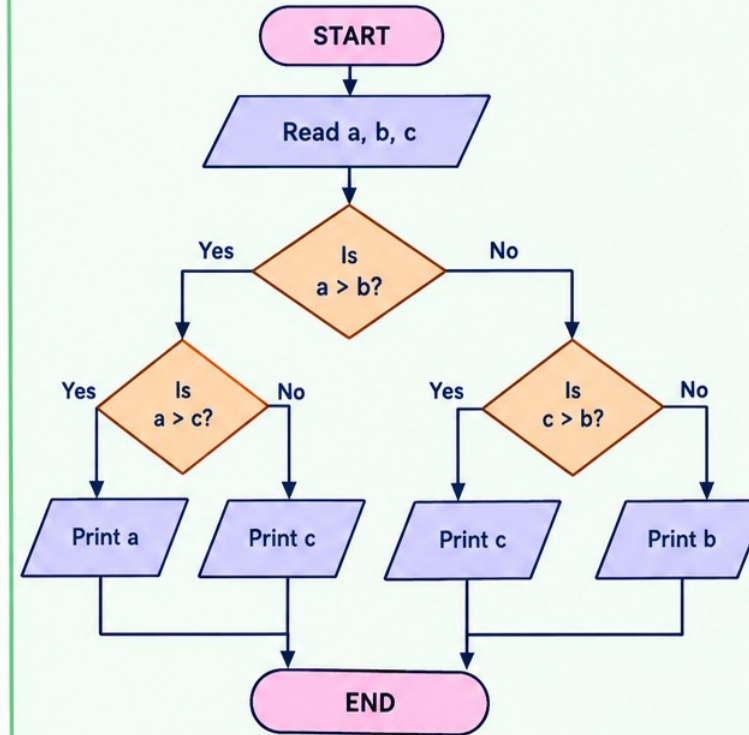
Symbols Used:



Logic:

The flowchart reads N numbers one by one, adds each number to S, and repeats until $I = N$. Finally, it prints the total sum.

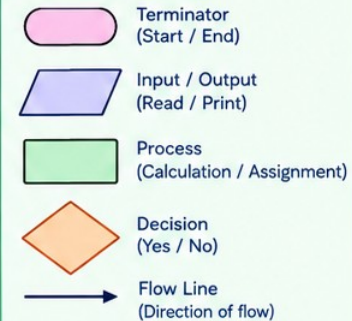
Example 26: Flowchart to Choose the Largest of Three Distinct Numbers



Steps:

1. Start the program.
2. Read values of a, b, c.
3. Check if $a > b$.
 - If Yes, check if $a > c$.
 - Yes $\rightarrow$ Print a
 - No $\rightarrow$ Print c
 - If No, check if $c > b$.
 - Yes $\rightarrow$ Print c
 - No $\rightarrow$ Print b
4. End the program.

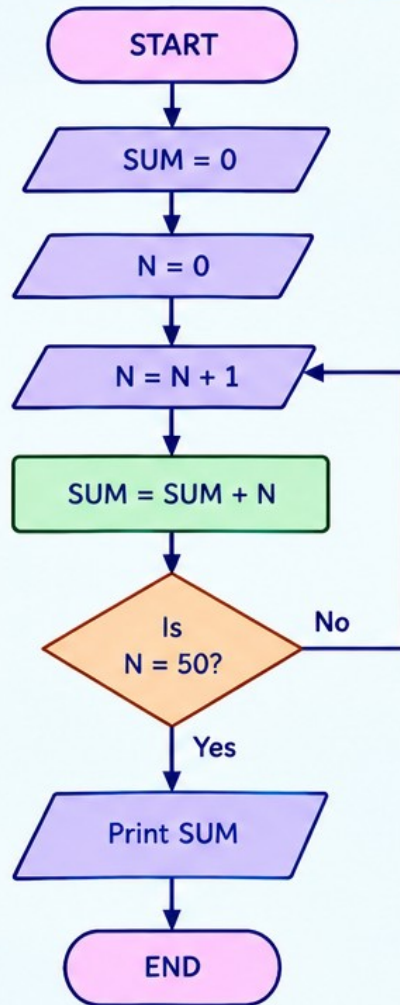
Symbols Used:



Logic:

The flowchart compares the three numbers using decision symbols and prints the largest number.

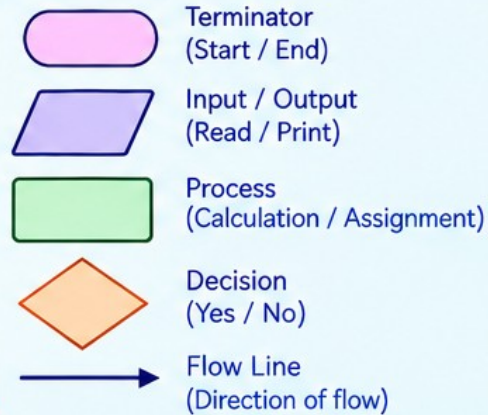
Example 27 : Design a flowchart to draw a flowchart to find the sum of first 50 natural numbers.



Steps:

1. Start the program.
2. Set SUM = 0.
3. Set N = 0.
4. Increment N ($N = N + 1$).
5. Add N to SUM ($SUM = SUM + N$).
6. Check if N = 50.
 - If No, repeat from step 4.
 - If Yes, go to step 7.
7. Print the value of SUM.
8. End the program.

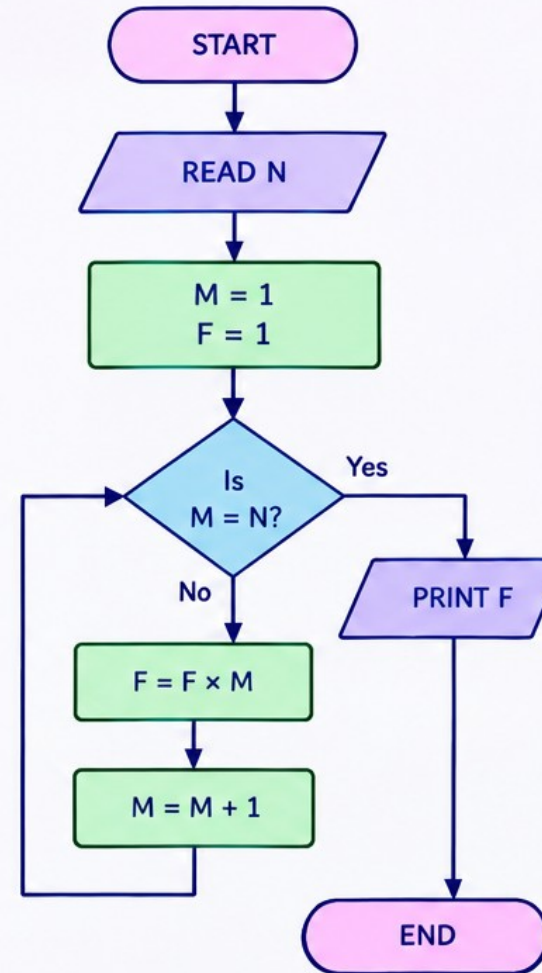
Symbols Used:



Logic:

The flowchart uses a loop to add numbers from 1 to 50. It keeps adding each number to SUM and finally prints the total sum.

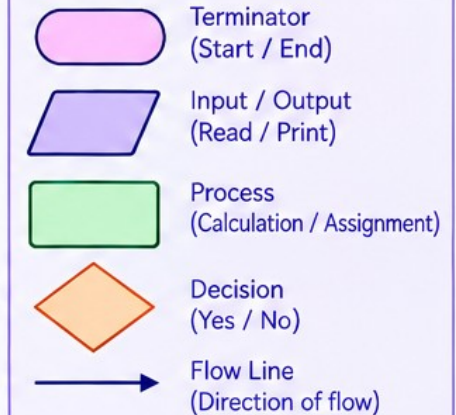
Example 28 : Design a flowchart for the computation of factorial N (N!).



Steps:

1. Start the program.
2. Read the value of N.
3. Set M = 1 and F = 1.
4. Check if M = N.
 - If Yes, go to step 7.
 - If No, go to step 5.
5. Multiply F by M ($F = F \times M$).
6. Increment M ($M = M + 1$).
7. Print the value of F.
8. End the program.

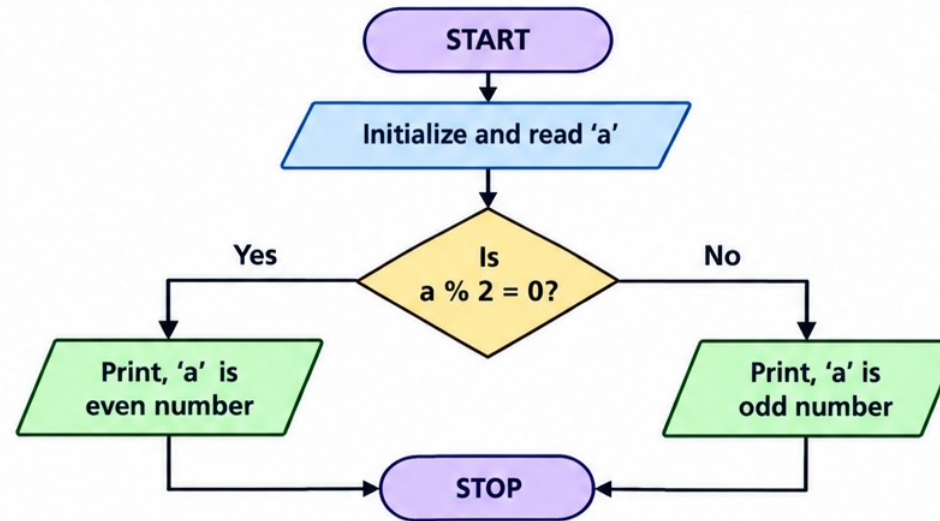
Symbols Used:



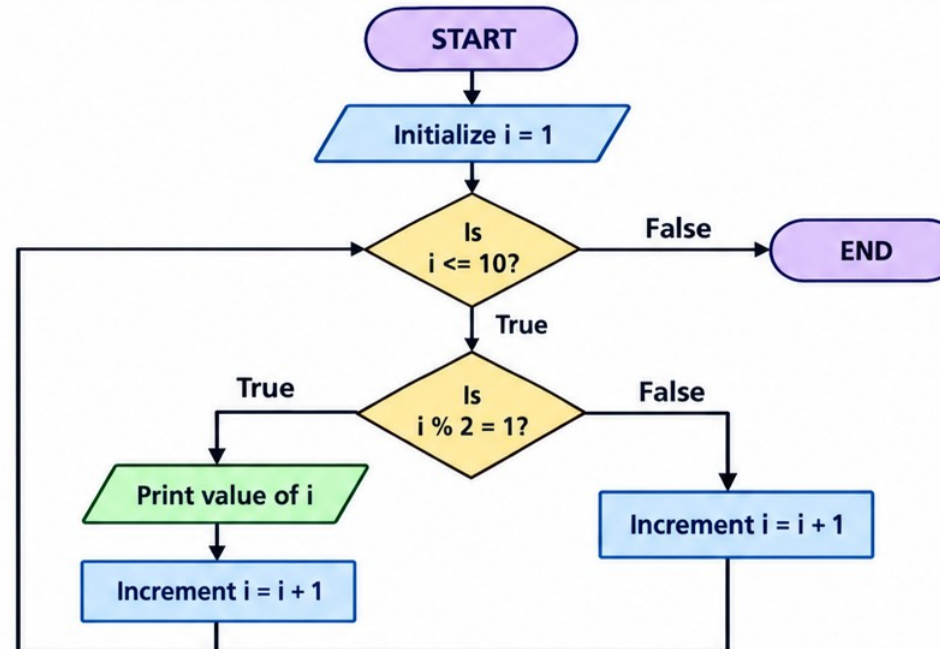
Logic:

The flowchart multiplies numbers from 1 to N using a loop. The final value of F gives the factorial of N (N!).

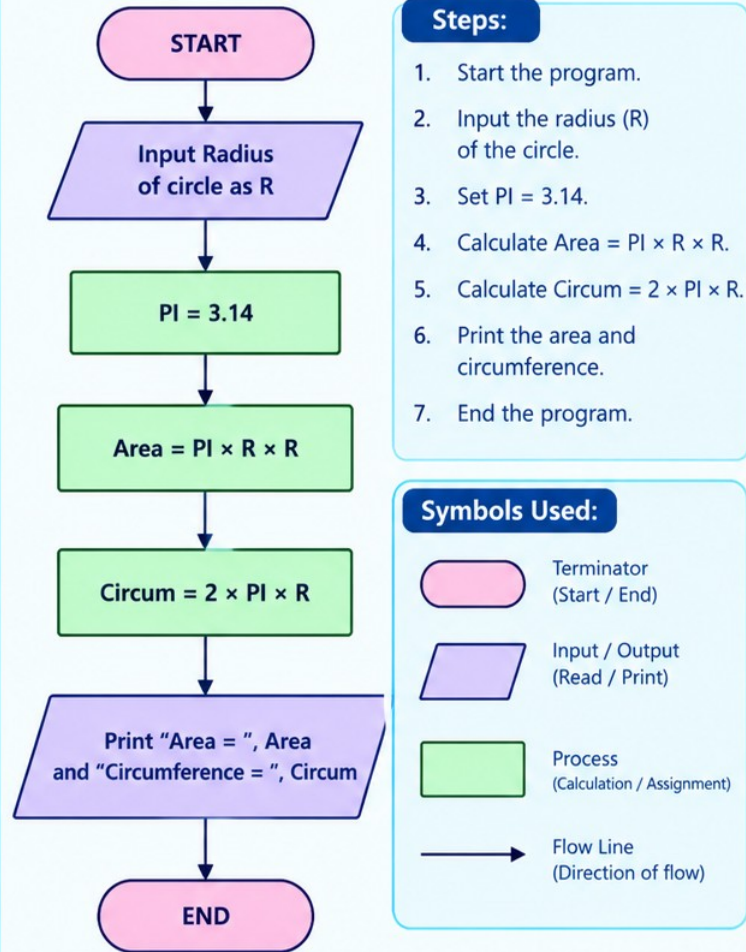
Example 29: Determine whether a number is even or odd



Example 30: Draw a flowchart to print first 10 odd numbers



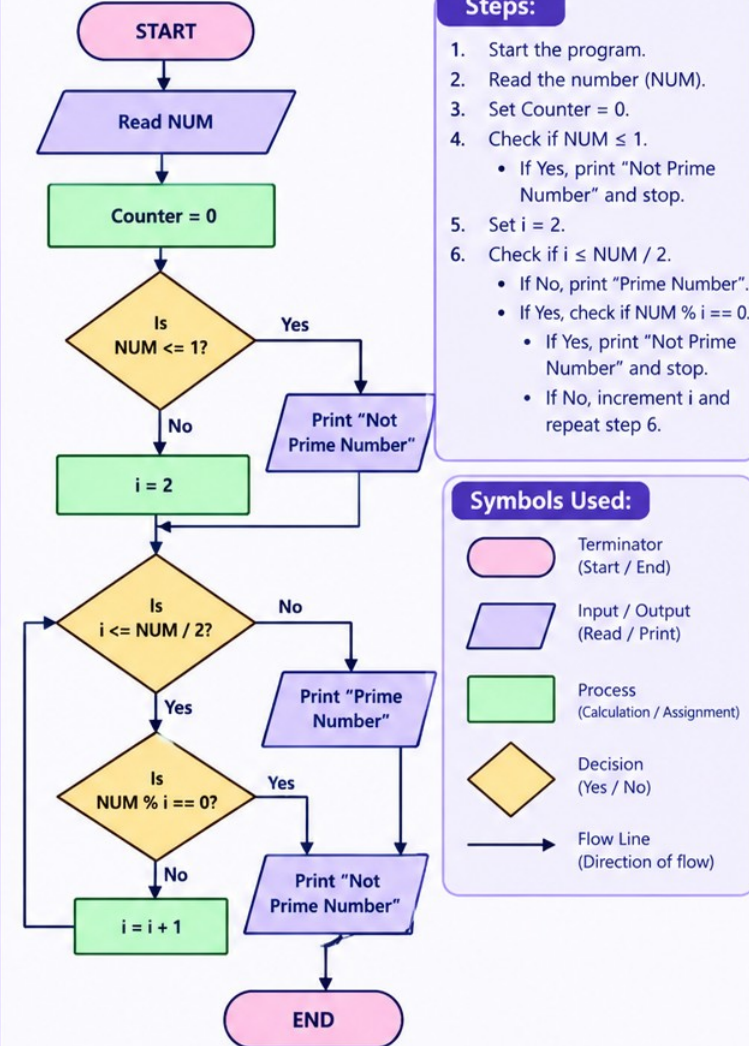
Example 31: Draw a flowchart to find the area and circumference of a circle.



Logic:

The flowchart takes the radius of the circle, calculates the area and circumference using the formulas and then prints the results.

Example 32: Draw a flowchart to check prime number.



Logic:

The flowchart checks whether the number is less than or equal to 1. If not, it divides the number by all numbers from 2 to $n/2$. If any number divides it exactly, it is not prime. Otherwise, it is a prime number.

Unit 2 : Algorithm & Flowchart

Quick Revision



1 Algorithm

Algorithm is a finite, step-by-step procedure used to solve a problem.

Example (Sum of two numbers)

1. Start
2. Read A, B
3. $S = A + B$
4. Print S
5. Stop

3 Ways of Writing an Algorithm

1. Natural Language
2. Step-form
3. Pseudocode
4. Flowchart

2 Characteristics of Algorithm

F-D-I-O-F

- F** Finiteness – Must end after finite steps.
- D** Definiteness – Every step must be clear.
- I** Input – Takes zero or more inputs.
- O** Output – Produces at least one output.
- F** Effectiveness – Each step must be simple and executable.

4 Step-form

Algorithm written in numbered sequential steps is called step-form.

Example:

1. Start
2. Input two numbers
3. Add the numbers
4. Display result
5. Stop

5 Pseudocode

Pseudocode is an informal way of writing an algorithm using simple English-like statements.

Example:

```
START
READ A, B
SUM ← A + B
PRINT SUM
STOP
```

Conventions:

- Use simple English.
- Use meaningful variable names.
- One instruction per line.
- Use indentation.
- Use IF, ELSE, WHILE, etc.
- Avoid language-specific syntax.

6 Flowchart

A flowchart is a graphical representation of an algorithm or program flow. It uses standard symbols to represent different steps of a problem. It helps in understanding, testing and debugging a program.

Uses:

- Understand program logic
- Find logical errors
- Testing and debugging
- Program documentation
- Easy communication
- Helps in maintenance

7 Flowchart Symbols

Symbol	Name	Use
	Terminator	Start / End
	Process	Calculation / Action
	Data I/O	Input / Output
	Decision	Yes / No
	Flow Line	Direction of flow
	Connector	Connect flow
	Predefined Process	Sub-process

★ Easy Trick:

Oval = Start/Stop, Rectangle = Process, Parallelogram = I/O, Diamond = Decision

8 Guidelines for Drawing Flowchart

1. Arrange requirements in logical order.
2. Keep it clear and easy to follow.
3. Flow should be top-to-bottom or left-to-right.
4. One flow line from a process symbol.
5. Decision has two or more paths.
6. Terminal symbol has one incoming/outgoing flow.
7. Use standard symbols.
8. Avoid unnecessary line intersections.
9. Logical start and finish.
10. Test with sample data.

9 Advantages of Flowchart

- C** Communication – Easy to communicate logic.
- A** Analysis – Makes analysis easier.
- D** Documentation – Useful for documentation.
- D** Debugging – Helps find logical errors.
- M** Maintenance – Easier maintenance.

10 Disadvantages of Flowchart

- **Complex Logic** – Large programs make it complicated.
- **Alteration** – Changes may require redrawing.
- **Reproduction** – Difficult to reproduce manually.
- **Time-consuming** – Drawing takes time.
- **Subjective** – Different programmers may draw differently.

★ Important Examples (22 – 32)

22 Making a Cup of Tea
(See previous image)

23 Retirement Age
(See previous image)

24 ATM
(See previous image)

25 Sum of N Numbers
(See previous image)

26 Largest of 3 Numbers
(See previous image)

27 Sum of First 50 Natural Numbers
(See previous image)

28 Factorial of N
(See previous image)

29 Even or Odd
(See previous image)

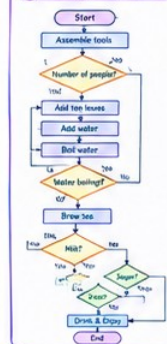
30 First 10 Odd Numbers
(See previous image)

31 Area & Circumference of Circle
(See previous image)

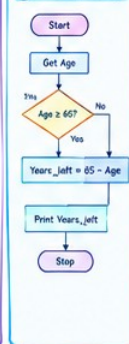
32 Prime Number
(See previous image)

✓ Remember: Practice all flowcharts for exam!

22 Making a Cup of Tea



23 Retirement Age



24 ATM



25 Sum of N Numbers



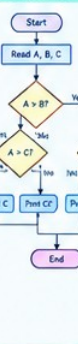
26 Largest of 3 Numbers



27 Sum of First 50 Natural Numbers



28 Factorial of N



29 Even or Odd



30 First 10 Odd Numbers



31 Circle



32 Prime Number

